Faculty of : FCE Program: B.Tech	**Class/Section: 3rd Year 5th Sem C & E**	**Date: ……**
Name of Faculty: Dr. Kriti Sankhla	**Name of Course: Advance Java**	**Code: BCECCE5104**

Unit-4 Hibernate

Hibernate Framework

Hibernate is a framework which provides some **abstraction layer**, meaning that the programmer does not have to worry about the implementations, Hibernate does the implementations for you internally like **Establishing a connection with the database, writing query to perform CRUD operations etc**. Hibernate is a Java framework that simplifies the development of Java application to interact with the database.

It is a **java framework** which is used to develop persistence logic. Persistence logic means to store and process the data for long use. More precisely Hibernate is an open-source, non-invasive, light-weight java ORM (Object-relational mapping) framework to develop objects which are independent of the database software and make independent persistence logic in all JAVA, JEE.

Framework means it is special install-able software that provides an abstraction layer on one or more technologies like JDBC, Servlet, etc to simplify or reduce the complexity for the development process.

Open-Source means:

- Hibernate framework is available for everyone without any cost.
- The source code of Hibernate is also available on the Internet and we can also modify the code.

Light-weight means:

- Hibernate is less in size means the installation package is not big is size.
- Hibernate does not require any heavy container for execution.
- It does not require POJO and POJI model programming.
- Hibernate can be used alone or we can use Hibernate with other java technology and framework.

Non-invasive means:

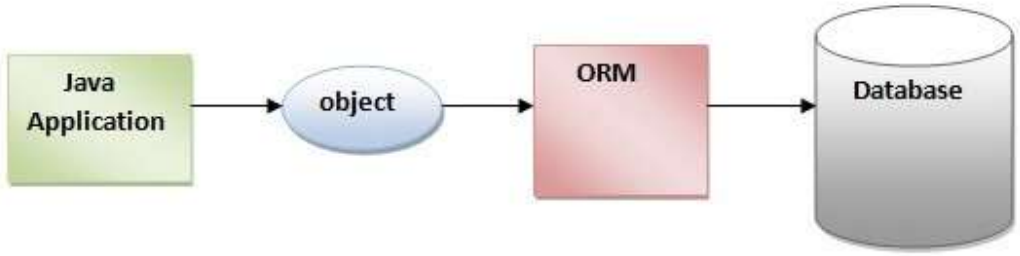
- The classes of Hibernate application development are loosely coupled classes with respect to Hibernate API i.e. Hibernate class need not implement hibernate API interfaces and need not extend from Hibernate API classes.

Functionalities supported by Hibernate framework

- Hibernate framework support **Auto DDL** operations. In JDBC manually we must create table and declare the data-type for each column. But Hibernate can do **DDL operations** for you internally like creation of table,drop a table,alter a table etc.
- Hibernate supports **Auto Primary key generation**. It means in JDBC we must manually set a primary key for a table. But Hibernate can this task for you.
- Hibernate framework is independent of Database because it supports **HQL (Hibernate Query Language)** which is not specific to any database, whereas JDBC is database dependent.
- In Hibernate, **Exception Handling is not mandatory**, whereas In JDBC exception handling is mandatory.
- Hibernate supports **Cache Memory** whereas JDBC does not support cache memory.
- Hibernate is a **ORM tool** means it support Object relational mapping. Whereas JDBC is not object oriented moreover we are dealing with values means primitive data. In hibernate each record is represented as a Object but in JDBC each record is nothing but a data which is nothing but primitive values.

ORM Tool

An ORM tool simplifies the data creation, data manipulation and data access. It is a programming technique that maps the object to the data stored in the database.



The ORM tool internally uses the JDBC API to interact with the database.

JPA

Java Persistence API (JPA) is a Java specification that provides certain functionality and standard to ORM tools. The javax.persistence package contains the JPA classes and interfaces.

Advantages of Hibernate Framework

1) Open Source and Lightweight

Hibernate framework is open source under the LGPL license and lightweight.

2) Fast Performance

The performance of hibernate framework is fast because cache is internally used in hibernate framework. There are two types of cache in hibernate framework first level cache and second level cache. First level cache is enabled by default.

3) Database Independent Query

HQL (Hibernate Query Language) is the object-oriented version of SQL. It generates the database independent queries. So you don't need to write database specific queries. Before Hibernate, if database is changed for the project, we need to change the SQL query as well that leads to the maintenance problem.

4) Automatic Table Creation

Hibernate framework provides the facility to create the tables of the database automatically. So there is no need to create tables in the database manually.

5) Simplifies Complex Join

Fetching data from multiple tables is easy in hibernate framework.

6) Provides Query Statistics and Database Status

Hibernate supports Query cache and provide statistics about query and database status.

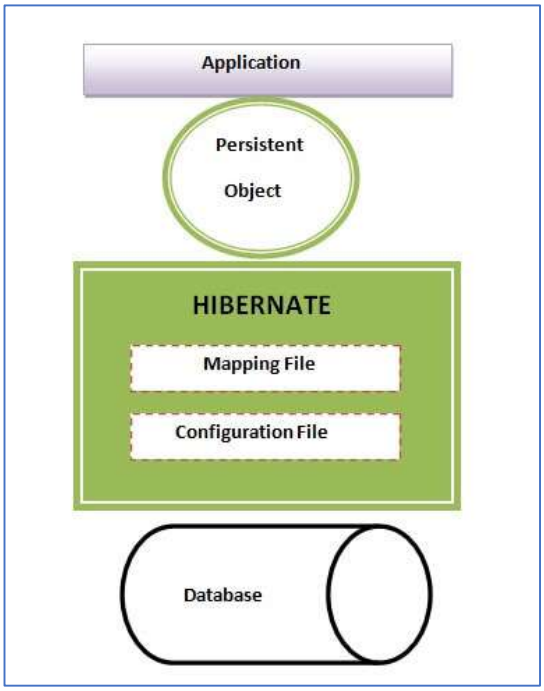
Hibernate Architecture

The Hibernate architecture includes many objects such as persistent object, session factory, transaction factory, connection factory, session, transaction etc.

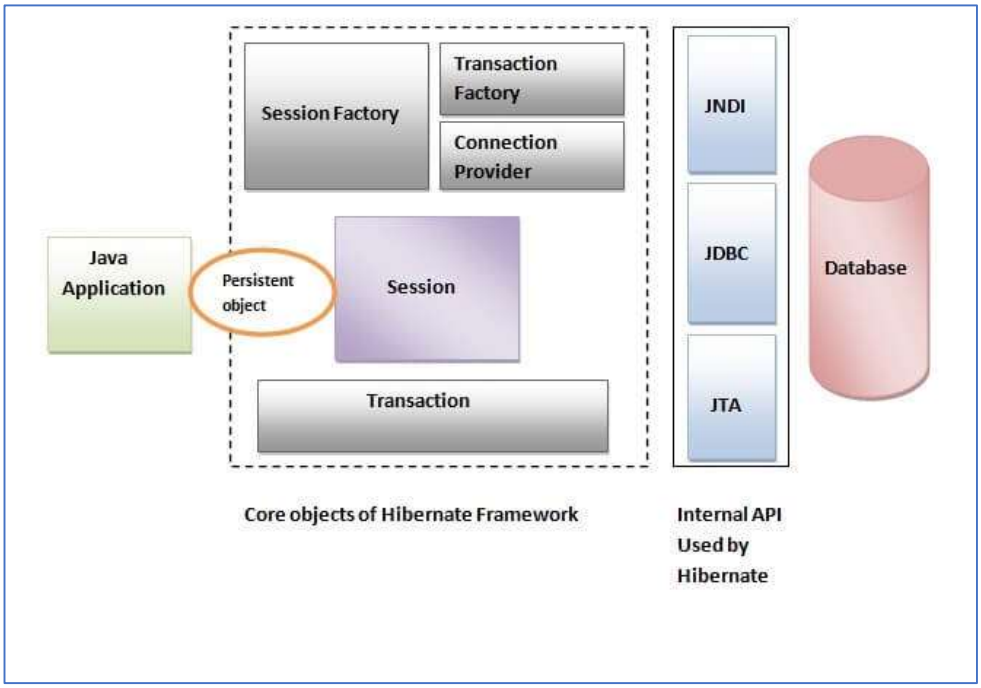
The Hibernate architecture is categorized in four layers.

- Java application layer
- Hibernate framework layer
- Backhand api layer
- Database layer

Let us see the diagram of hibernate architecture:



This is the high level architecture of Hibernate with mapping file and configuration file.



Hibernate framework uses many objects such as session factory, session, transaction etc. alongwith existing Java API such as JDBC (Java Database Connectivity), JTA (Java Transaction API) and JNDI (Java Naming Directory Interface).

Elements of Hibernate Architecture

SessionFactory

The SessionFactory is a factory of session and client of ConnectionProvider. It holds second level cache (optional) of data. The org.hibernate.SessionFactory interface provides factory method to get the object of Session.

Session

The session object provides an interface between the application and data stored in the database. It is a short-lived object and wraps the JDBC connection. It is factory of Transaction, Query and Criteria. It holds a first-level cache (mandatory) of data. The org.hibernate.Session interface provides methods to insert, update and delete the object. It also provides factory methods for Transaction, Query and Criteria.

Transaction

The transaction object specifies the atomic unit of work. It is optional. The org.hibernate.Transaction interface provides methods for transaction management.

ConnectionProvider

It is a factory of JDBC connections. It abstracts the application from DriverManager or DataSource. It is optional.

TransactionFactory

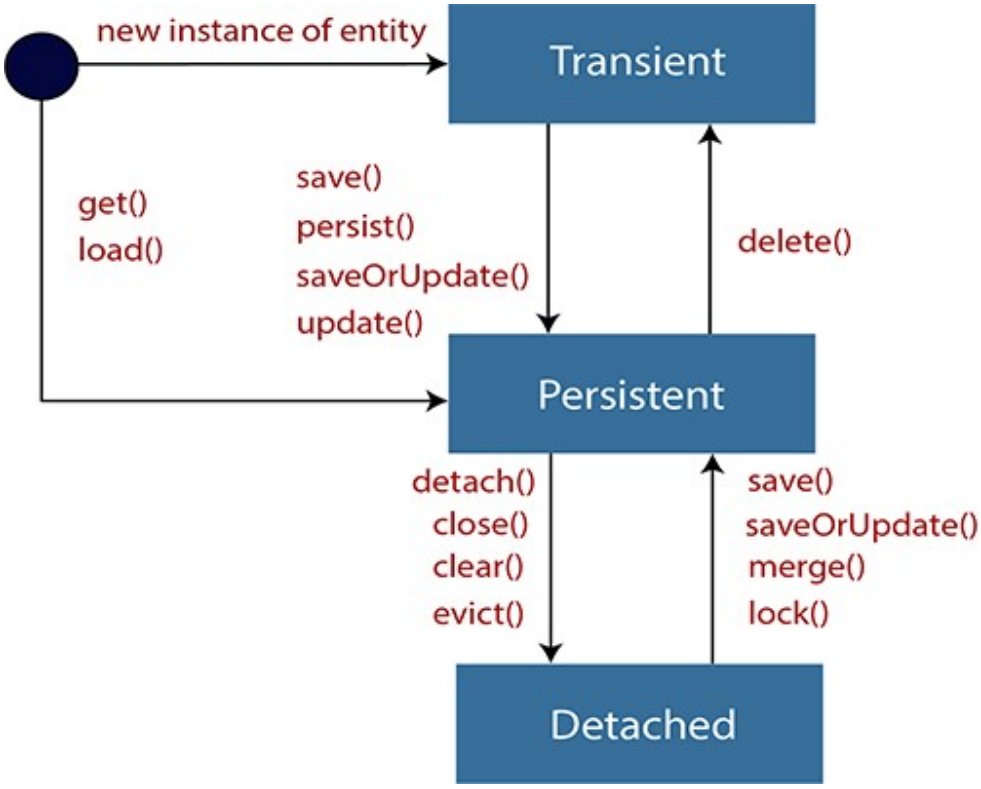
It is a factory of Transaction. It is optional.

Hibernate Lifecycle

In Hibernate, either we create an object of an entity and save it into the database, or we fetch the data of an entity from the database. Here, each entity is associated with the lifecycle. The entity object passes through the different stages of the lifecycle.

The Hibernate lifecycle contains the following states: -

- Transient state
- Persistent state
- Detached state



Transient state

- The transient state is the initial state of an object.
- Once we create an instance of POJO class, then the object entered in the transient state.
- Here, an object is not associated with the Session. So, the transient state is not related to any database.
- Hence, modifications in the data don't affect any changes in the database.
- The transient objects exist in the heap memory. They are independent of Hibernate.

1. Employee e=**new** Employee(); //Here, object enters in the transient state.
2. e.setId(101);
3. e.setFirstName("Gaurav");

4. e.setLastName("Chawla");

Persistent state

- As soon as the object associated with the Session, it entered in the persistent state.
- Hence, we can say that an object is in the persistence state when we save or persist it.
- Here, each object represents the row of the database table.
- So, modifications in the data make changes in the database.

We can use any of the following methods for the persistent state.

1. session.save(e);
2. session.persist(e);
3. session.update(e);
4. session.saveOrUpdate(e);
5. session.lock(e);
6. session.merge(e);

Detached State

- Once we either close the session or clear its cache, then the object entered into the detached state.
- As an object is no more associated with the Session, modifications in the data don't affect any changes in the database.
- However, the detached object still has a representation in the database.
- If we want to persist the changes made to a detached object, it is required to reattach the application to a valid Hibernate session.
- To associate the detached object with the new hibernate session, use any of these methods - load(), merge(), refresh(), update() or save() on a new session with the reference of the detached object.

We can use any of the following methods for the detached state.

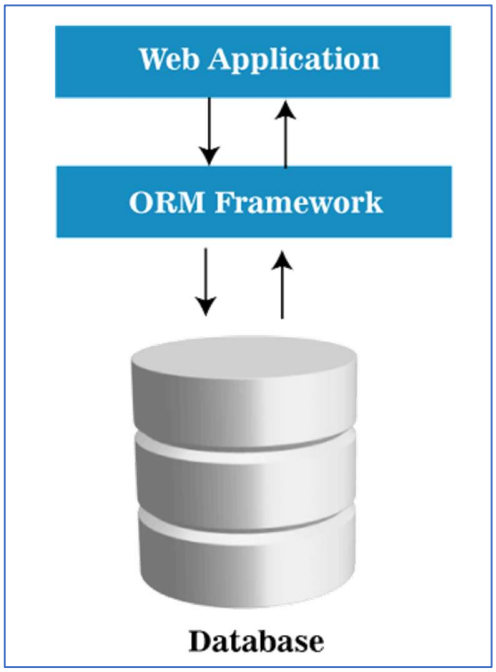
1. session.close();
2. session.clear();
3. session.detach(e);
4. session.evict(e);

ORM Tools in Java

ORM stands for **Object Relation Mapping.** It is a middleware application or tool that sits between the web application and database. It wraps the implementation specific details of storage drivers in an API.

What is ORM?

ORM is a technique for converting data between Java objects and relational databases (table). In simple words, we can say that the **ORM implements responsibility of mapping the object to relational model and vice-versa**. the ORM tool does mapping in such a way that model class becomes a table in the database and each instance becomes a row of the table.



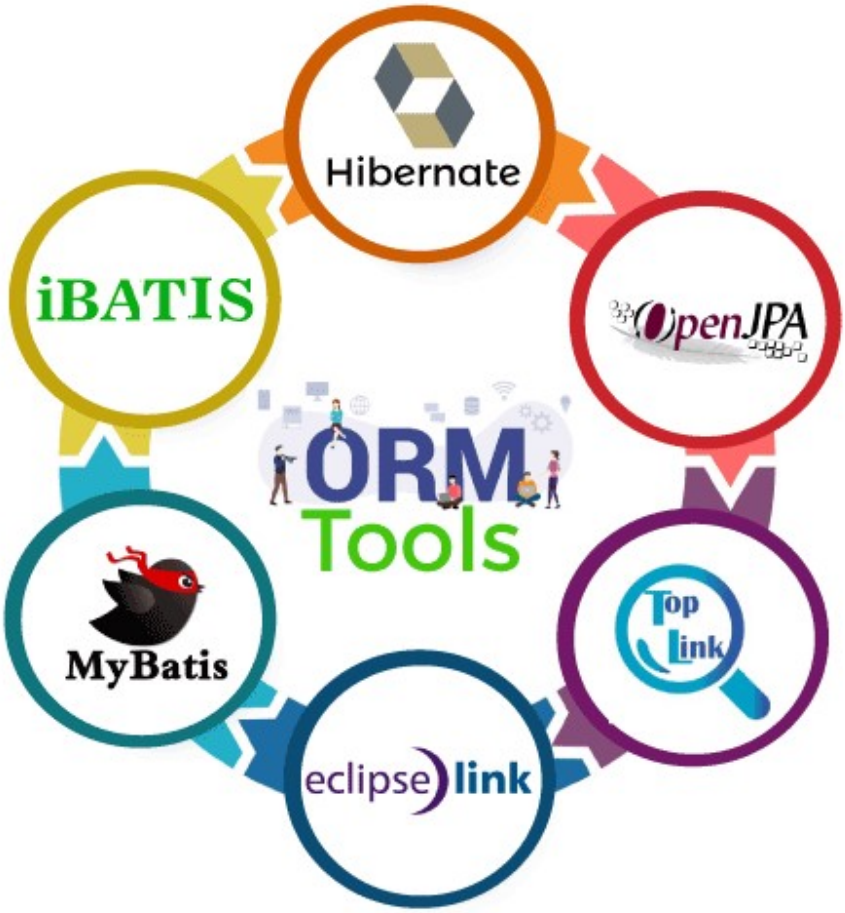
Advantages of ORM Tool

- It saves time and efforts.
- It gives pace to development process.
- It reduces the development cost.
- It provides connectivity to the database.
- It makes development more object-oriented.
- Easy transaction management.
- No need to implement database manually.
- Modification in any model (object or relational model) does not affect each other.

ORM Tools

There are many ORM tools available but the following ORM tools are the most commonly used.

- Hibernate
- TopLink
- EclipseLink
- OpenJPA
- MyBatis (formally known as iBatis)



Hibernate

Hibernate is a Java persistence framework that simplifies the development of Java application to interact with the database. It is an open source, widely used, lightweight, ORM tool. Hibernate implements the specifications of JPA (Java Persistence API) for data persistence.

Advantages of Hibernate

- It is an open source, lightweight, flexible, and powerful ORM tool.
- It supports database independent query (HQL).
- Its performance is fast because cache is internally used.
- It simplifies the complex join.
- If there is any modification in table or database only need to change in XML file properties.

TopLink

TopLink is an ORM tool developed by Oracle especially for Java developers. It is a persistence framework that is a part of Oracle's OracleAS, WebLogic, OC4J servers. It provides development tools and run-time functionalities that ease the development process and increases the functionality. Persistent object data is stored in relational databases that helps build high-performance applications. Storing data in either XML (Extensible Markup Language) or relational databases is made possible by transforming it from object-oriented data.

Advantages of TopLink

- Provides GUI interface for defining mapping.
- More flexible mapping types.
- Independence between object model and SQL schema.
- It provides development flexibility like code business logic into model, developer can deal with an existing model.
- It provides more flexible mapping types.

OpenJPA

Apache OpenJPA is a Java persistence project at The Apache Software Foundation that can be used as a stand-alone POJO persistence layer or integrated into any Java EE compliant container and many other lightweight frameworks, such as Tomcat and Spring.

Advantages of OpenJPA

- Declarative mapping.
- It can be use as stand-alone POJO persistence layer.
- It can be easily integrated into any EJB complaint container.

MyBatis

MyBatis was formerly known as iBatis. It is also an open source persistence framework that simplifies the implementation of database. Basically, it performs the following four things:

- It executes SQL safely and abstracts away all the intricacies of JDBC.
- It maps parameter objects to JDBC prepared statement parameters.
- It maps rows in JDBC result sets to objects.

It is different from other ORM tools. The main difference between MyBatis and other ORM tool is that MyBatis emphasis the use of SQL while other ORM tools uses custom query language (HQL).

Advantages of MyBatis

- It provides tags instead of using logic code.
- It is quite flexible and close to JDBC.
- Separation of SQL statements form code.
- It supports inline SQL.
- Maintenance is easy because it encapsulates SQL in the form of stored procedure.
- It also supports dynamic SQL.

EclipseLink

EclipseLink is the open source Eclipse Persistence Services tool introduced by Eclipse Foundation. It is an extensible framework that allows Java developers to interact with various data services such as databases, web services, Object XML mapping, and enterprise information systems. It supports the following four persistence standards:

1. JPA (Jakarta Persistence)
2. JAXB (Jakarta XML Bindings)
3. JCA (Jakarta Connectors)
4. SDO (Service Data Object)

Advantages of EclipseLink

- It is based on TopLink, so it is easy to use if someone is already friendly with TopLink.
- EclipseLink fully supports usage in an OSGi environment.

Basic Program in Hibernate

Components of Hibernate

A Basic Hibernate program consists of mainly five major components:

- **Hibernate jar files:** Hibernate jar files need to be included in the project. **Hibernate Configuration File:** It consists of configurations required to run hibernate like database URL, username, password, etc.
- **POJO (Plain Old Java Object) class:** It defines the entity which is to be mapped into the database.
- **Hibernate Mapping Class:** It provides the structure for creating the schema in the database corresponding to the entity.
- **Main Class:** It defines the code to be executed.

Hibernate – Query Language

Hibernate is a Java framework that makes it easier to create database-interactive Java applications. In HQL, instead of a table name, it uses a class name. As a result, it is a query language that is database-independent.

Hibernate converts HQL queries into SQL queries, which are used to perform database actions. Although Native SQL may be used directly with Hibernate, it is encouraged to utilize HQL wherever feasible to prevent database portability issues.

HQL has many benefits. Some benefits are:

- HQL is database-independent.
- polymorphic queries supported which are type-safe.
- It is portable and easy to learn for Java programmers.

The **Query interface** provides object-oriented methods and capabilities for representing and manipulating HQL queries.

Note: 1. *Keywords like FROM, SELECT, WHERE are* ***not case-sensitive*** *in HQL.*
2. Table and Column names are ***case-sensitive*** *in HQL.*

Hibernate Query Language (HQL) Clauses

There are many HQL clauses available to interact with relational databases, and several of them are listed below:

1. FROM Clause
2. SELECT Clause
3. WHERE Clause
4. ORDER BY Clause
5. UPDATE Clause
6. DELETE Clause
7. INSERT Clause

For details and examples regarding each clause is mentioned below.

1. FROM Clause

To load a whole persistent object into memory, the FROM clause is used.

```
String hib = "FROM Student";

Query query = session.createQuery(hib);
List results = query.list();
```

2. SELECT Clause

The SELECT clause is used when only a *few attributes of an object* are required rather than the entire object.

```
String hib = "SELECT S.roll FROM Student S";

Query query = session.createQuery(hib);
List results = query.list();
```

3. WHERE Clause

Filtering records is done with the WHERE clause. It's used to *retrieve only the records that meet a set of criteria*.

```
String hib = "FROM Student S WHERE S.id = 5";

Query query = session.createQuery(hib);
List results = query.list();
```

4. ORDER BY Clause

The ORDER BY clause is used to *sort the results* of an HQL query.

```
String hib = "FROM Student S WHERE S.id > 5 ORDER BY S.id DESC";

Query query = session.createQuery(hib);
List results = query.list();
```

NOTE:

1. ***Order By – DESC*** *will sort in descending order*

2. ***Order By – ASC*** *will sort in ascending order*

5. UPDATE Clause

The UPDATE clause is required to *update the value* of an attribute.

```
String hib = "UPDATE Student set name=:n WHERE roll=:i";

Query q=session.createQuery(hib);
q.setParameter("n","John");
q.setParameter("i",23);

int status=q.executeUpdate();
System.out.println(status);
```

6. DELETE Clause

It is required to *delete a value* of an attribute.

```
String hib = "DELETE FROM Student WHERE id=10";

Query query=session.createQuery(hib);
query.executeUpdate();
```

7. INSERT Clause

It is required to *Insert values* into the relation.

```
String hib = "INSERT INTO Student(first_name, last_name)" +
            "SELECT first_name, last_name FROM backup_student";

Query query = session.createQuery(hib);
int result = query.executeUpdate();
```

Pagination using Query

For pagination using query we have **two** methods available for it, below table contains the methods and its description.

Method	**Action Performed**
Query setMaxResults(int max)	Instructs Hibernate to get a specific number of items.
Query setFirstResult(int starting_no)	Takes an integer as an argument that represents the first row in your result set, beginning with row 0.

Example for pagination using HQL

To fetch the few row data at a time we can use pagination using HQL. For this example we are fetching the rows 5 to 10.

```
String hib = "FROM Student"

Query query=session.createQuery(hib);
query.setFirstResult(5);
query.setMaxResult(10);

List list=query.list();
```

The above example returns the list of records from 5 to 10.

Aggregate Methods

Similar to SQL, HQL has several aggregation techniques, some of them are mentioned below-

- Average
- Max
- Min
- Count
- Sum

For details and examples regarding each aggregate method is mentioned below.

Average

To find the average of ***marks***

```
String hib = "SELECT AVG(marks) FROM Student";
Query q=session.createQuery(hib);
```

Max

To find the maximum *mark* among all the available marks.

```
String hib = "SELECT MAX(marks) FROM Student";
Query q=session.createQuery(hib);
```

Min

To find the minimum *mark* among all the available marks.

```
String hib = "SELECT MIN(marks) FROM Student";
Query q=session.createQuery(hib);
```

Count

To find the count of *id* present.

```
String hib = "SELECT COUNT(id) FROM Student";
Query q=session.createQuery(hib);
```

NOTE: F*or counting only distinct values you can use DISTINCT in the following ways-*
1. With the SELECT clause as "SELECT DISTINCT name FROM Student"
*2. With the FROM clause (deprecated) as "SELECT * FROM DISTINCT Student"*

Sum

To find the sum of all the marks of Student.

```
String hib = "SELECT SUM(marks) FROM Student";

Query q=session.createQuery(hib);

List<Integer> list=q.list();
System.out.println(list.get(0));
```

Difference Between ORM and JDBC

ORM when compared to JDBC is easier to work with as it does all the work by itself. It maps Java classes to the database variables via XML. While working with domain-driven applications and in the case of complex object relationships, ORM is mostly preferred but when the application is simple enough then it is better to use JDBC.

Object Relational Mapping	**Java Database Connectivity**
Little slower than JDBC	It is faster compared to ORM
SQL queries requirement is comparatively quite less however this doesn't mean that you have to do less work using ORM	SQL queries are required here
Hibernate framework (working on ORM technology) makes it easy to store objects/data to database automatically without writing manual code	We have to write code manually to store objects/ data in the database
The flow from Object/data to hibernate i.e. the frontend part is based on the ORM technique	Whereas when the data is stored in the database finally i.e., the backend part is still based on JDBCin
There are not many restrictions while dealing with data. Even a single database cell can be retrieved, changed, and saved.	JDBC comes with a lot of restrictions on extracting the result-set, processing it, and then committing it back to the database.

Hibernate – Annotations

Annotation in JAVA is used to represent supplemental information. Here are some annotations used in our POJO specifically for hibernate-

Annotations	**Use of annotations**
@Entity	Used for declaring any POJO class as an entity for a database

@Table	Used to change table details, some of the attributes are- • name – override the table name • schema • catalogue • enforce unique constraints
@Id	Used for declaring a primary key inside our POJO class
@GeneratedValue	Hibernate automatically generate the values with reference to the internal sequence and we don't need to set the values manually.
@Column	It is used to specify column mappings. It means if in case we don't need the name of the column that we declare in POJO but we need to refer to that entity you can change the name for the database table. Some attributes are- • Name – We can change the name of the entity for the database • length – the size of the column mostly used in strings • unique – the column is marked for containing only unique values • nullable – The column values should not be null. It's marked as NOT
@Transient	Tells the hibernate, not to add this particular column
@Temporal	This annotation is used to format the date for storing in the database
@Lob	Used to tell hibernate that it's a large object and is not a simple object
@OrderBy	This annotation will tell hibernate to OrderBy as we do in SQL. For example – we need to order by student firstname in ascending order @OrderBy("firstname asc")

These are some annotations that are mostly used in order to work with hibernate.

Student.java (POJO class)

```
import javax.persistence.*;
@Entity
@Table(name="student")
public class Student {
   @Id
@GeneratedValue(Stretagy=”GeneratedType.IDENTITY)
   private int id;
   private String firstName;
   private String city;
   public int getId() {
      return id;
   }
   public void setId(int id) {
      this.id = id;
   }
   public String getFirstName() {
      return firstName;
   }
   public void setFirstName(String firstName) {
      this.firstName = firstName;
   }
```

```
public String getCity() {
    return city;
}
 public void setCity(String city) {
    this.city = city;
}
}
```

Persistent Classes in Hibernate

In Hibernate, a **persistent class** is an ordinary Java class (also known as a **POJO** - Plain Old Java Object) that is mapped to a table in a database. Each instance of a persistent class represents a row in the database table.

Key Characteristics of Persistent Classes:

- They must have a **default constructor** (without parameters).
- The class must be **serializable** (though this is optional, it's recommended in a distributed environment).
- Each persistent class must have a field (or a set of fields) that uniquely identifies each instance, typically called a **primary key**. This is mapped using the @Id annotation or a similar configuration in XML mapping.
- Fields in the persistent class should be **private** or **protected**, with public **getter and setter methods**.

Example: Persistent Class (Product)

Let's create a persistent class called Product, which represents a product in a database.

```
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String name;
    private String category;
    private double price;

    // Default constructor
    public Product() {}

    // Parameterized constructor
    public Product(String name, String category, double price) {
        this.name = name;
        this.category = category;
        this.price = price;
```

```
}

// Getters and Setters
public int getId() {
   return id;
}

public void setId(int id) {
   this.id = id;
}

public String getName() {
   return name;
}

public void setName(String name) {
   this.name = name;
}

public String getCategory() {
   return category;
}
```

```
    public void setCategory(String category) {
        this.category = category;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    // Override toString() method for better display
    @Override
    public String toString() {
        return "Product{" +
                "id=" + id +
                ", name='" + name + '\'' +
                ", category='" + category + '\'' +
                ", price=" + price +
                '}';
    }
}
```

Hibernate – Types of Mapping

In Hibernate, **mapping types** are used to map Java types to SQL types. These mappings allow Hibernate to translate Java objects into a form that the database understands, and vice versa. Hibernate provides several built-in mapping types for primitive data types, objects, collections, and more complex relationships like one-to-many and many-to-many.

Types of Mapping in Hibernate

1. **Basic Types (Primitive and Wrapper types)**
2. **Custom Types (User-Defined)**
3. **Collection Types**
4. **Component Types**
5. **Entity Relationships**

1. Basic Types (Primitive and Wrapper Types)

These are simple mappings between Java data types and their corresponding SQL types. Hibernate can automatically handle most primitive types and their corresponding wrapper types.

Java Type	Hibernate Type	SQL Type
int	integer	INTEGERw
long	long	BIGINT
short	short	SMALLINT
float	float	FLOAT
double	double	DOUBLE
boolean	boolean	BIT
char	character	CHAR

String	string	VARCHAR
Date	date, time, timestamp	DATE, TIME, TIMESTAMP
BigDecimal	big_decimal	DECIMAL
BigInteger	big_integer	NUMERIC
byte[]	binary	BINARY or VARBINARY
Serializable	serializable	Database-dependent

Example: Mapping Basic Types

Here's how you would map basic types in a Product entity:

```
@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(name = "name", nullable = false)
    private String name;

    @Column(name = "price", nullable = false)
    private double price;

    @Column(name = "category", nullable = false)
```

```
    private String category;

    @Column(name = "created_date")
    @Temporal(TemporalType.DATE)
    private Date createdDate;

    // Getters and Setters...
}
```

In this example:

- id, name, price, and category are mapped to SQL types like INTEGER, VARCHAR, etc.
- The createdDate is mapped using the @Temporal annotation to specify that only the date portion of the Date field should be stored.

2. Custom Types (User-Defined)

In some cases, the default Hibernate mapping types may not suffice, and you might need to define a custom type. Hibernate allows you to define custom types using either the @Type annotation or by implementing the UserType or CompositeUserType interfaces.

Example: Custom Type using @Type

Let's assume you have a Status enum in your application, and you want to map it to a VARCHAR field in the database.

```
public enum Status {
    ACTIVE, INACTIVE, SUSPENDED;
}

@Entity
```

```
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(name = "status")
    @Enumerated(EnumType.STRING)
    private Status status;

    // Getters and Setters...
}
```

- @Enumerated(EnumType.STRING) tells Hibernate to store the Status enum as a VARCHAR rather than its ordinal value (EnumType.ORDINAL would store the ordinal position of the enum).

You can also create fully custom types by implementing UserType or CompositeUserType interfaces, but this is generally more advanced.

3. Collection Types

Hibernate provides mapping support for collection types, such as List, Set, Map, and Bag. These are used to map relationships between entities, such as one-to-many and many-to-many relationships.

Supported Collection Types:

- **List**: Ordered collection that allows duplicate entries.

- **Set**: Unordered collection that does not allow duplicates.
- **Map**: Collection that associates keys to values.

Example: One-to-Many Mapping (Using List)

```
@Entity
@Table(name = "category")
public class Category {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;
   @Column(name = "name")
   private String name;
   @OneToMany(mappedBy = "category", cascade = CascadeType.ALL)
   private List<Product> products;
   // Getters and Setters...
}
@Entity
@Table(name = "products")
public class Product {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private int id;
   @Column(name = "name")
   private String name;
```

```
    @ManyToOne
    @JoinColumn(name = "category_id")
    private Category category;
   // Getters and Setters...
}
```

Here:

- Category has a one-to-many relationship with Product, so a List<Product> is mapped.
- The @OneToMany annotation is used to define this relationship.

4. Component Types

Component types (also called **Embeddable** types) are used when you want to map multiple columns in a table to a single class that is not an entity. This is useful for grouping related fields together.

Example: Embeddable Address Type

```
@Embeddable
public class Address {
    @Column(name = "street")
    private String street;
    @Column(name = "city")
    private String city;
    @Column(name = "zipcode")
    private String zipcode;
```

```
    // Getters and Setters...
}

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Embedded
    private Address address;

    // Getters and Setters...
}
```

Here, Address is an embeddable type that will map its fields (street, city, and zipcode) to corresponding columns in the users table.

5. Entity Relationships

a. One-to-One

Maps a one-to-one relationship between two entities.

```
@OneToOne(cascade = CascadeType.ALL)
```

```
@JoinColumn(name = "profile_id")
private Profile profile;
```

b. One-to-Many / Many-to-One

Defines a one-to-many or many-to-one relationship between entities.

```
@OneToMany(mappedBy = "category")
private List<Product> products;
@ManyToOne
@JoinColumn(name = "category_id")
private Category category;
```

c. Many-to-Many

Maps a many-to-many relationship between two entities, often requiring a join table.

```
@ManyToMany
@JoinTable(
   name = "student_course",
   joinColumns = @JoinColumn(name = "student_id"),
   inverseJoinColumns = @JoinColumn(name = "course_id")
)
private List<Course> courses;
```

Hibernate – Native SQL

Hibernate allows you to execute **native SQL queries** (i.e., raw SQL) directly against the database using its **Native SQL API**. This is useful when you want to execute complex SQL queries that cannot

be expressed using HQL (Hibernate Query Language) or when you want to use database-specific features.

Why Use Native SQL in Hibernate?

- **Database-specific queries**: You may want to use vendor-specific SQL functions or syntax that HQL does not support.
- **Complex queries**: Sometimes, the query is too complex to express using HQL or the criteria API.
- **Optimization**: Native queries can sometimes offer performance benefits by allowing you to write optimized SQL.

Key Components for Native SQL in Hibernate

1. **Session.createSQLQuery()**:
    - This method is used to create an SQL query.
2. **Result Transformers**:
    - You can transform the result of the query into an entity using setResultTransformer() to automatically map the result set to an entity.
3. **Entity Mapping**:
    - If you're querying data that can be mapped directly to a Hibernate entity, Hibernate can automatically handle the conversion from SQL results to entity objects.

HCQL (Hibernate Criteria Query Language)

The Hibernate Criteria Query Language (HCQL) is used to fetch the records based on the specific criteria. The Criteria interface provides methods to apply criteria such as retreiving all the records of table whose salary is greater than 50000 etc.

Advantage of HCQL

The HCQL provides methods to add criteria, so it is **easy** for the java programmer to add criteria. The java programmer is able to add many criteria on a query.

Criteria Interface

The Criteria interface provides many methods to specify criteria. The object of Criteria can be obtained by calling the **createCriteria()** method of Session interface.

Syntax of createCriteria() method of Session interface

1. **public** Criteria createCriteria(Class c)

The commonly used methods of Criteria interface are as follows:

1. **public Criteria add(Criterion c)** is used to add restrictions.
2. **public Criteria addOrder(Order o)** specifies ordering.
3. **public Criteria setFirstResult(int firstResult)** specifies the first number of record to be retreived.
4. **public Criteria setMaxResult(int totalResult)** specifies the total number of records to be retreived.
5. **public List list()** returns list containing object.
6. **public Criteria setProjection(Projection projection)** specifies the projection.

Restrictions class

Restrictions class provides methods that can be used as Criterion. The commonly used methods of Restrictions class are as follows:

1. **public static SimpleExpression lt(String propertyName,Object value)** sets the **less than** constraint to the given property.
2. **public static SimpleExpression le(String propertyName,Object value)** sets the **less than or equal** constraint to the given property.
3. **public static SimpleExpression gt(String propertyName,Object value)** sets the **greater than** constraint to the given property.
4. **public static SimpleExpression ge(String propertyName,Object value)** sets the **greater than or equal** than constraint to the given property.

5. **public static SimpleExpression ne(String propertyName,Object value)** sets the **not equal** constraint to the given property.
6. **public static SimpleExpression eq(String propertyName,Object value)** sets the **equal** constraint to the given property.
7. **public static Criterion between(String propertyName, Object low, Object high)** sets the **between** constraint.
8. **public static SimpleExpression like(String propertyName, Object value)** sets the **like** constraint to the given property.

Order class

The Order class represents an order. The commonly used methods of Restrictions class are as follows:

1. **public static Order asc(String propertyName)** applies the ascending order on the basis of given property.
2. **public static Order desc(String propertyName)** applies the descending order on the basis of given property.

Examples of Hibernate Criteria Query Language

There are given a lot of examples of HCQL.

Example of HCQL to get all the records

1. Crietria c=session.createCriteria(Emp.**class**);//passing Class class argument
2. List list=c.list();

Example of HCQL to get the 10th to 20th record

1. Crietria c=session.createCriteria(Emp.**class**);
2. c.setFirstResult(10);
3. c.setMaxResult(20);
4. List list=c.list();

Example of HCQL to get the records whose salary is greater than 10000

1. Crietria c=session.createCriteria(Emp.**class**);
2. c.add(Restrictions.gt("salary",10000));//salary is the propertyname
3. List list=c.list();

SessionFactory

The SessionFactory in Hibernate is a factory for Session objects. It is responsible for creating and managing the lifecycle of Session instances. A SessionFactory is typically created once during application initialization and is a **thread-safe** object.

Key Characteristics:

- **Heavyweight Object**: The SessionFactory is a heavyweight object because it holds database connection details and is created only once for the entire application.
- **Thread-safe**: You can share a single SessionFactory across threads because it is designed to be thread-safe.
- **Configuration**: It reads Hibernate configuration files (like hibernate.cfg.xml or hibernate.properties) to establish connections to the database.
- **Caching**: The SessionFactory provides caching mechanisms, which help in improving application performance by avoiding unnecessary database hits.

Creation:

A SessionFactory is created using a configuration object or by reading a configuration file. This is usually done when the application starts up.

SessionFactory sessionFactory = new Configuration().configure().buildSessionFactory();

Usage:

Once created, you can use the SessionFactory to open sessions for interacting with the database:

Session session = sessionFactory.openSession();

Lifecycle:

- **Created once** per application, typically at startup.
- **Closed** when the application shuts down.

Session

A Session in Hibernate represents a single unit of work and is used to interact with the database. It acts as a wrapper around a JDBC connection and is **not thread-safe**. Each session is associated with a specific connection to the database.

Key Characteristics:

- **Lightweight Object**: Unlike SessionFactory, a Session is lightweight and meant to be short-lived.
- **Not Thread-safe**: A Session should not be shared between multiple threads.
- **Manages Transactions**: It is responsible for initiating and managing database transactions.
- **Manages Entity State**: It manages the lifecycle states of entities, including transient, persistent, and detached.
- **Caching**: The Session object also provides a first-level cache that stores entities loaded within the same session. This cache is enabled by default.

Common Operations:

- **Persisting entities**: Insert new rows into the database.
- **Updating entities**: Update existing rows.
- **Deleting entities**: Remove rows.
- **Querying**: Fetch data from the database using HQL, Criteria, or SQL queries.

Example:

```
Session session = sessionFactory.openSession();

Transaction tx = session.beginTransaction();

Product product = new Product();
```

```
product.setName("Laptop");
product.setPrice(800.0);
product.setCategory("Electronics");
session.save(product);  // Insert operation
tx.commit();
session.close();
```

Lifecycle of a Session:

1. **Open Session**: A session is opened using sessionFactory.openSession().
2. **Perform Operations**: Operations like save(), update(), delete(), or query() are performed.
3. **Transaction Management**: Start, commit, or rollback a transaction using session.beginTransaction(), transaction.commit(), and transaction.rollback().
4. **Close Session**: The session must be closed after the operations are completed using session.close(). This releases the database connection.

3. Transaction Management with Session

Transactions are critical in ensuring the **atomicity** of a database operation. In Hibernate, transactions are handled using the Session object:

```
Session session = sessionFactory.openSession();
Transaction transaction = null;
try {
    transaction = session.beginTransaction();
    // Perform database operations
    session.save(product);
    transaction.commit();  // Commit the transaction if successful
```

```
} catch (Exception e) {
    if (transaction != null) {
        transaction.rollback();  // Rollback if there's an exception
    }
    e.printStackTrace();
} finally {
    session.close();  // Always close the session
}
```

4. Session vs SessionFactory

Aspect	Session	SessionFactory
Scope	Per transaction/request	Per application (created once per application)
Thread Safety	Not thread-safe	Thread-safe
Creation/Destruction	Opened per transaction/request and closed after use	Created once at application startup and closed at application shutdown
Purpose	Used for performing CRUD operations, managing transactions	Used for creating Session objects and caching configurations

5. Caching in Session and SessionFactory

- **First-level cache (Session)**: This is associated with the session object and is used to cache entities for the duration of the session. The first-level cache is mandatory and cannot be disabled.
- **Second-level cache (SessionFactory)**: This is associated with the SessionFactory and can be shared among multiple sessions. Second-level caching must be explicitly configured using third-party libraries like EHCache or OSCache.